Genetic Programming for Spectrum Based Fault Localization

by Sebastian Wærling (20266301) on November 9, 2026

1 Introduction

Fault Localization (FL) can be a useful technique for developers to locate the source of a failure based on test results. One approach is Spectrum-Based Fault Localization (SBFL), which utilizes a spectrum of four integers:

- `ep`: # test cases that execute unit and pass
- `ef`: # test cases that execute unit and fail
- `np`: # test cases that do not execute unit and pass
- `nf`: # test cases that do not execute unit and fail

Based on the spectrum it is possible to obtain a suspiciousness score according to some risk evaluation formula. If a program unit receives a higher score, it is more likely to be the suspected source of failure. Multiple human-designed risk evaluation formulas have been suggested over time based on intuition and thought Yoo (2012).

An automated way of generating solutions, in this case risk evaluation formulas, is to apply Genetic Programming (GP). Inspired by Charles Darwin's theory of evolution, solutions can inherit from each other, mutate, and be selected for future generations. The next section will describe how GP can be used to find a suitable risk evaluation formula for SBFL.

2 GP Implementation

Overall, the GP implementation is highly inspired by the one applied in Yoo (2012). The following steps, except for the initialization of the population, are repeated for 100 iterations. For each iteration, the next generation is chosen to be the offsprings of the former generation, i.e., generational replacement is applied.

2.1 Population Initialization

The first step is to initialize a population of risk evaluation formulas. Each formula is represented as a tree whose non-terminal nodes can be any of the following arithmetic operators:

- Addition (`add`)
- Subtraction (`sub`)
- Multiplication (`mul`)
- Division (`div`)
- Square Root (`sqrt`)

The `div` operator handles zero division by returning 1 if the denominator is equal to 0. Similarly, the `sqrt` operator takes the square root of the absolute value of the input to avoid dealing with complex numbers. The leaf nodes can be any of the spectrum components: `ep`, `ef`, `np`, `nf`, or the integer 1.

The trees in the initial population are being grown according to the ramped half and half method, which means that half of the population is grown using the *full* method while the other half is grown using the *grow* method. A maximum tree depth of 4 is being applied to all the generated trees and the initial population size is 40.

2.2 Fitness Evaluation

Every generation including the initial population, is evaluated in terms of accuracy@1 (`acc@1`) on the training bugs, i.e., the proportion of bugs ranked first according to the corresponding risk evaluation formula. The formulas in the given generation are then ranked according to their `acc@1`. This ranking is used for selecting parents in the next step.

2.3 Selection of Fitter Individuals

The rank selection operator is applied to sample two parents for offspring creation. This operator assigns a probability of selection to each individual in the current population. A linear scheme can be described by the following formula:

$$P_{\text{linear}}(i) = \frac{2-s}{N} + \frac{r_i(s-1)}{\sum_{j=1}^{N} r_j}, \quad 1 \leq s \leq 2 \quad (1)$$

where s is a smoothing hyperparameter, N is the size of the population, and r_i is the rank of individual i. The ranks range from 0 to $N-1$. Alternatively, one can also apply an exponential schema:

$$P_{\text{exp}}(i) = \frac{1-e^{-r_i}}{\sum_{j=1}^{N} 1-e^{-r_j}}. \quad (2)$$

In this implementation both schemas are applied. For the the linear schema, s is set to 1.5 for balance.

2.4 Offspring Creation

After sampling two parents according to the probabilities computed by (1), two offsprings (children) are created according to the single point crossover operator. This operator chooses a random cutpoint in the two parent trees. The resulting subtrees are then swapped to generate two children trees. The sampling and crossover process is repeated 20 times such that a total of 40 offsprings are created, which maintains the initial population size. A rate between 0 and 1 can be set such that the parents are not guaranteed to be cut. Inspired by Yoo (2012), this rate is set to 1.0 such that the crossover is performed for all sampled parent pairs.

2.5 Mutation

Subtree replacement mutation is applied to each of the generated offsprings with rate 0.08. The rate implies that an offspring is mutated with probability 0.08. Similar to the single point crossover operator, a random cutpoint is chosen. The resulting subtree is then replaced by a new random subtree, which is generated in the same manner as during population initialization. During mutation it is ensured that the mutated tree still has a maximum depth of 4.

3 Experimental Setup

The raw coverage data are collected from Just and Defects4J contributors. The related failing tests and buggy lines are split unevenly across four projects; `Chart`, `Lang`, `Math`, and `Time`. A stratified train/test-split is made to obtain a hold-out test set for subsequent evaluation. The split ratio is 70/30 in favor of the training set, which is used to evaluate the fitness of any generation during the GP iterations. The stratified split ensures that the proportion of each of the four projects is preserved from the full dataset.

Due to the stochasticity of population initialization, selection of parents, crossover, and mutation, the GP loop is repeated 30 times, as in Yoo (2012). For each run, the best performing formula in terms of `acc@1` is saved. After the 30 runs, the candidate formulas are evaluated on a hold-out test set, and the formula with the highest `acc@1` on those test bugs is chosen as the final formula. For reproducibility purposes, each run is seeded with the run ID, such that the same run always results in the same candidate formulae being saved.

For the final evaluation, the chosen risk evaluation formula is assessed in terms of `acc@1` on all 119 bugs. Additionally, the wasted effort (`wef`) will be computed across the four projects. This metric corresponds to the average rank of the buggy program element. Thus, the `wef` should be as low as possible.

4 Results & Discussion

The best performing risk evaluation formula turned out to be (3) below:

$$\frac{2e_f - 1}{\sqrt{2n_f + 2e_p}}. \tag{3}$$

Intuitively, it makes sense to have e_f in the numerator, as a higher number of failing tests that executed the given program unit should be proportional to the risk of it being a buggy line. Consequently, when e_p is high, the risk should be lower, i.e., it makes sense to have e_p in the denominator. If n_f is high, it might indicate that the buggy line is somewhere else, why it also makes intuitive sense to have it in the denominator. The square root reduces the impact of a change in either n_f or e_p, so the main driver for the risk score is e_f.

Across all 119 bugs, the formula achieves an `acc@1` of 40.3%. The `wef` on each of the four projects can be seen in Table 1 below. In general, the `wef` is low, especially for the `Lang` and `Math` projects with larger support.

Project	`wef`	n
`Chart`	6.63	19
`Lang`	4.82	34
`Math`	5.40	48
`Time`	8.33	18

Table 1: Wasted effort (`wef`) for GP generated risk evaluation formula as well as the support (n) across four different projects.

The GP implementation offers many opportunities for adjustments and experiments. One of them is the rank selection operator, which is based on the rank of the individual formulas rather than their `acc@1`. If two individuals achieve almost the same `acc@1`, they should have almost equal probabilities of being selected as parents, but the rank-

ing implies that all individuals are equally distant from each other.

To further improve the `acc@1` of the final formula, some of the hyperparameters, such as the smoothing s, the max depth, or the mutation rate, could have been tuned to gain optimal performance. However, the results are promising and the `wef` scores in Table 1 suggests that GP can be used to generate a risk evaluation formula that limits the search space for buggy lines to just a few lines of code.

References

R. Just and Defects4J contributors. Defects4J: A Database of Real Faults to Enable Controlled Testing Studies for Java Programs. GitHub repository. URL https://github.com/rjust/defects4j. Accessed: 2026-11-01.

S. Yoo. Evolving Human Competitive Spectra-Based Fault Localisation Techniques. In G. Fraser and J. Teixeira de Souza, editors, *Search Based Software Engineering (SSBSE 2012)*, volume 7515 of *Lecture Notes in Computer Science*, pages 244–258, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-33119-0_18. URL https://doi.org/10.1007/978-3-642-33119-0_18.